Fraud Detection Pipeline

System documentation and usage guide

Remittance transaction fraud scoring • Internal technical documentation

This system turns raw MongoDB transaction exports and partner chargeback sheets into a pseudonymised training table, trains a calibrated gradient-boosted model on it, and exposes a scoring interface that returns a fraud probability with explanations.

It does **not** make approve / review / block decisions. Those live in a separate decisioning system. This service answers one question: how likely is this transaction to be fraud, and why.

> **Two properties everything else is built around**
>
> **1. Security by exclusion.** Sensitive fields are dropped, never hashed into the output. The schema contract is an allowlist, so a new sensitive field added upstream cannot leak in: no code path reads it.
>
> **2. Point-in-time correctness.** Every feature for a transaction at time T is computable using only information available at T. Rolling windows exclude the current row; only immutable fields are trusted on historical records.

1. Quickstart

1.1 Prerequisites

Python 3.11 or newer. Git. Optionally Docker, for a local MongoDB when testing the database source path.

1.2 Install

```
git clone <your-repo> fraud-pipeline
cd fraud-pipeline

python -m venv .venv
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\Activate.ps1         # Windows PowerShell

pip install -r requirements.txt
```

1.3 Configure secrets

Create a `.env` file in the project root. It is gitignored and must never be committed.

```
FRAUD_SALT=<a long random string>
```

Generate one once, and **never change it**. The salt makes pseudonymisation deterministic, which is what allows a chargeback sheet to join to a transfer. Changing it invalidates every hash already produced.

```
python -c "import secrets; print(secrets.token_hex(32))"
```

1.4 Generate test data

For local development, synthesise a dataset rather than using real records. It produces all five pipeline inputs with correct referential integrity, all four payment-provider variants, append-only flag histories, and a chargeback sheet whose join keys resolve.

```
python tools/generate_synthetic.py --n-transfers 100000
```

1.5 Run the pipeline

```
python src/run_pipeline.py --chargebacks data/partner_chargebacks.csv

# useful flags
python src/run_pipeline.py --limit 1000      # small sample first
python src/run_pipeline.py --skip-profile    # skip schema profiling
```

Output: `data/training_set.parquet`, plus intermediate tables under `data/extracted/`.

1.6 Train a model

```
python src/train.py --trials 20
```

Writes `final_xgb.json`, `calibrator.joblib`, `model_prep.joblib`, `training_results.json` and `cost_grid.json` to `artifacts/`.

1.7 Run the tests

```
pytest                    # everything (~1 min: generates data, runs pipeline)
pytest -m "not slow"      # fast unit tests only (~1 second)
pytest -m security        # leakage and PII checks
```

1.8 Score a transaction

```
import joblib, xgboost as xgb
from model_prep import ModelPrep
import inference

prep  = joblib.load("artifacts/model_prep.joblib")
model = xgb.XGBClassifier(enable_categorical=True)
model.load_model("artifacts/final_xgb.json")
cal   = joblib.load("artifacts/calibrator.joblib")

X = prep.transform(transaction_df)        # SAME preprocessor as training
result = inference.score(model, cal, X.iloc[[0]],
                         txn=transaction_df.iloc[0].to_dict(),
                         booster=model.get_booster(),
                         feature_order=model.get_booster().feature_names)

print(result.to_dict())
# {'probability': 0.83, 'risk_band': 'High', 'risk_factors': [...], ...}
```

2. Architecture

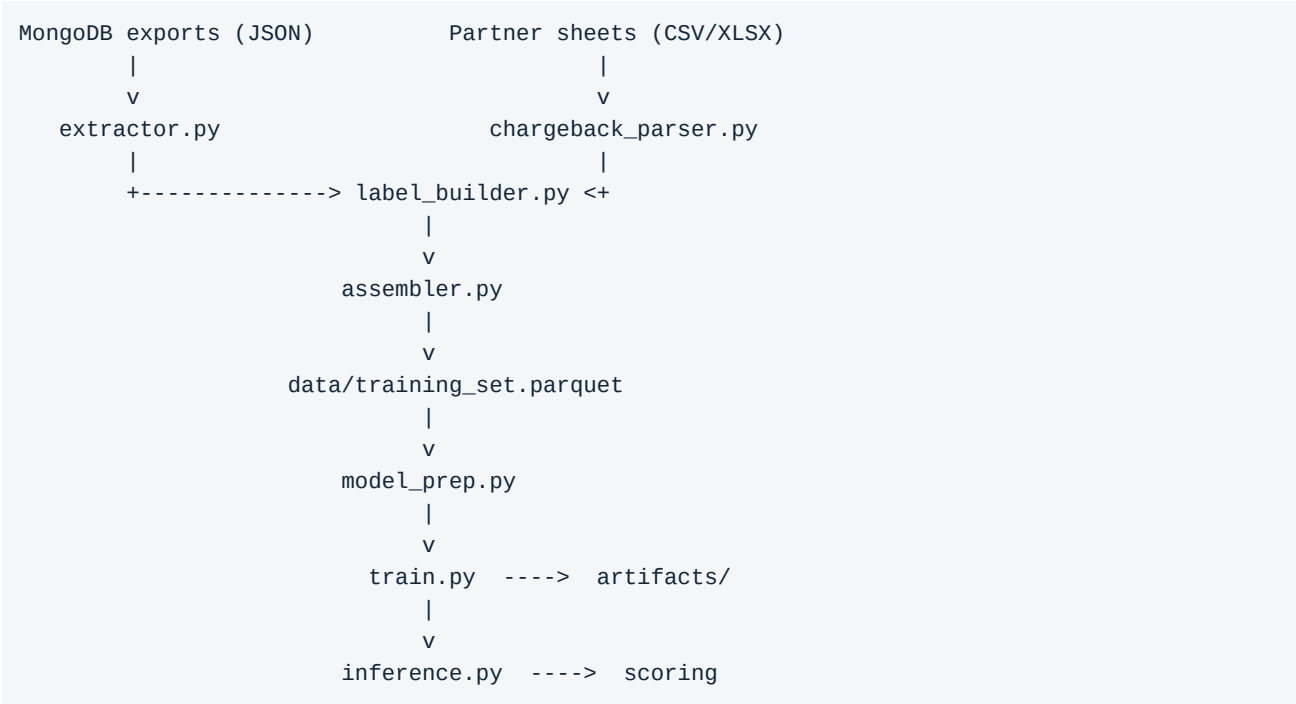
The pipeline runs in five stages, each reading the previous stage's output. `schema_contract.py` is the single source of truth every stage consults.

Stage	Module	Does
A	`profile_schema.py`	Detects provider-driven schema variance. Run on every fresh export.
B	`extractor.py`	Raw JSON to flat tables. Applies the allowlist, hashes join keys, resolves canonical fields.
C0	`chargeback_parser.py`	Partner CSV/XLSX to confirmed-fraud labels. Optional.
C	`label_builder.py`	Flag history plus chargebacks to a per-transaction label.
D	`assembler.py`	Joins, derives features, computes behavioural aggregates.
E	`test_pipeline.py`	Safety assertions on the output. Not optional in spirit.

Modelling sits downstream of the pipeline:

Module	Does
`model_prep.py`	Bridge from training table to feature matrix: temporal split, frequency encoding, categorical dtypes. Fit on train only.
`metrics.py`	Ranking metrics, threshold metrics, and the money cost model. Schema-agnostic by design.
`train.py`	Optuna tuning, LightGBM challenger, isotonic calibration, validation-selected threshold.
`inference.py`	The scoring seam. The only entry point a UI or API should call.

2.1 Data flow



3. Configuration

3.1 Secrets (.env)

Key	Purpose	Notes
`FRAUD_SALT`	Salt for pseudonymising join keys	Set once, never change. Treat as seriously as the data itself.
`KAGGLE_USERNAME`	Optional, for public dataset downloads	Development only
`KAGGLE_KEY`	Optional	Development only

In AWS, prefer SSM Parameter Store over a file: it gives access control, encryption at rest, and an audit trail of who read the value.

3.2 Settings (config.py)

Key	Default	Purpose
`EXTRACTED_DIR`	`data/extracted`	Intermediate tables
`TRAINING_SET_PATH`	`data/training_set`	Assembler output

Key	Default	Purpose
`ARTIFACTS_DIR`	`artifacts`	Model artifacts
`VELOCITY_WINDOWS_HOURS`	`(1, 24, 168)`	Rolling aggregate windows
`FP_UNIT_COST`	`25.0`	Cost of one wrongly declined customer
`FN_LOSS_FRACTION`	`1.0`	Share of a fraudulent transfer actually lost
`SEED`	`42`	Reproducibility

4. Core concepts

The reasoning behind the design. Each of these prevents a failure mode that is silent rather than loud.

4.1 Security model: exclusion, not obfuscation

Sensitive fields — SSNs, password and PIN hashes, tokens, names, phone and account numbers, raw IP addresses, coordinates — are **dropped entirely**. They are never hashed into the output, because a hash of a low-entropy value is brute-forceable and shipping a derivative of a secret is still a liability.

Hashing is reserved for **non-sensitive join keys** that must stay linkable: account and transaction identifiers. These are salted SHA-256 pseudonyms used during aggregation and then dropped before training. The model never sees an identity.

> **Pseudonymous is not anonymous.** Anyone holding both the salt and the output could re-link. Keep the salt secret and treat the training set as a confidential internal asset.

4.2 The schema contract

`schema_contract.py` is declarative — it transforms nothing. Each field is classified on four axes:

Axis	Values
action	KEEP (a feature) • DERIVE (feeds a computed feature) • HASH (join key) • LABEL • META (pipeline logic only) • DROP
sensitivity	SENSITIVE (never reaches output) • INTERNAL
mutability	IMMUTABLE • STATIC (set once) • MUTABLE (changes over time) • NA
tier	1 (available on every record) • 2 (needs transfer-time snapshots)

It is an **allowlist**: any field not listed is dropped. This is what makes leakage structurally impossible rather than merely unlikely — there is no code path that reads an unlisted field.

4.3 Provider schema variance

`paymentProvider` is a **discriminator**: its value determines which other fields exist on the document. Across four provider variants, 29 of 86 fields vary. The partner-side transaction reference is named differently by each provider, so canonical fields coalesce the aliases:

```
partner_reference  <- omnexPaymentId | payment_receipt_number | transaction_id
partner_txn_id     <- omnexPaymentMtn | transaction_id
```

Both are extracted to maximise chargeback join coverage. Run `profile_schema.py` on every fresh export: new providers introduce new variants, and the extractor records a silent null for absent fields, which makes variance invisible unless you look for it.

4.4 Labels and the blind spot

Flag documents are **append-only**: every flag or unflag creates a new document, so a transfer may have a history. The **latest event by date** is the verdict.

Source	Label	Meaning
`confirmed_fraud_chargeback`	**1**	Partner-confirmed. Highest authority.
`confirmed_fraud_upheld`	**1**	Flagged, human review upheld it.
`confirmed_legit_reviewed`	**0**	Flagged, review cleared it. A clean negative.
`assumed_legit_never_flagged`	**0**	Never flagged. **Assumed**, not confirmed.

A chargeback **overrides** an unflag: if review cleared a transaction and the partner later charged it back, that is a false negative of the review process — the most valuable training example available.

> **The blind spot.** The assumed-legit bucket contains the fraud the rules never caught. Train on flag data alone and the model learns to *imitate the existing rules*, blind spots included. Partner chargeback data is what turns it into a rules-improver. The COVERAGE figure in the label audit is how much of your data has a confirmed verdict — watch it.

4.5 Point-in-time correctness

Every feature for a transaction at time T must use only information available at T. Three mechanisms enforce this:

- **Rolling windows use `closed="left"`.** This single parameter excludes a transaction from its own velocity aggregate. Without it the model sees a trace of the row it is predicting. A user's first transaction must show zero prior history — there is a test asserting exactly that.
- **Recipient history uses `cumcount()`**, counting prior sends only.
- **Only immutable fields are trusted on historical records.** User and recipient documents are not snapshotted at transfer time, so their current state may reflect changes caused by the very fraud being labelled. Creation dates are safe because they never change: account age and recipient age are correct even from a current snapshot.

4.6 Two-tier degradation

Tier	Available on	Examples
1	Every record	Amount, corridor, VPN/proxy/Tor, IP mismatch, account age, recipient age, 3DS, AVS
2	Future snapshotted records only	Account level, limit utilisation, verification state, billing-country match

Tier-2 columns are NaN on historical rows and populate automatically once transfer-time snapshots begin arriving. No code change is needed. This is deliberate graceful degradation: historical model strength is traded for correctness.

4.7 Calibration

`scale_pos_weight` handles class imbalance but inflates raw model scores, so an uncalibrated 0.6 is **not** a 60% fraud probability. Isotonic regression fitted on validation corrects this.

Because isotonic regression is monotonic, it does **not** change AUROC or PR-AUC. It is not a performance improvement — it fixes interpretability and downstream behaviour. That matters because a separate decisioning system thresholds on these numbers and a UI shows them to people. Never expose raw scores to a consumer that bands on them.

4.8 The cost model and threshold discipline

Ranking metrics say how well the model orders risk; they do not say where to draw the line. That is a business decision, and the cost function makes it explicit:

```
cost = (missed fraud, valued at the amount actually lost)
     + (wrongly declined customers x a parameterised friction cost)
```

Parameter	Meaning
`fp_unit_cost`	What one wrongly declined customer costs: support handling, lost margin, churn. Not measurable from data — expose it as a slider rather than asserting one figure.
`fn_loss_fraction`	What share of a fraudulent transfer the business actually eats. 1.0 (full principal) is usual for remittance, since funds are disbursed and rarely recovered.
`fn_fixed_cost`	Per-fraud overhead independent of amount: chargeback fees, investigation time.

> **Choose the threshold on validation, read test once.** Selecting it on test means minimising cost against the very data you then report. In measured testing that optimism was worth roughly **$1.09M per million transactions** — the difference between an honest number and one that collapses in production. The gap between the validation figure and the test figure IS the optimism. Trust the test figure.

4.9 Why PR-AUC, not just AUROC

At a ~2% fraud rate a model can post an excellent AUROC while being useless at the operating point you would actually run. PR-AUC's random baseline is the fraud rate itself, so it is reported alongside for context — 0.45 sounds poor until you note that random scores 0.02.

5. Testing

57 tests across three modules. Every failure mode they cover is silent: a leaked SSN raises no exception, a model trained on a leaked label just posts suspiciously good numbers, and a salt that loads differently produces zero chargeback matches that look like missing data.

Module	Covers
`test_metrics.py`	Cost arithmetic, threshold selection, ranking metrics, segment and grid reporting (16 tests)
`test_labels_and_inference.py`	Label precedence in isolation, risk bands, feature labelling, JSON safety (24 tests)
`test_security.py`	Hash determinism across processes, PII canary, metadata exclusion, point-in-time guards, row reconciliation (17 tests)

The most valuable three:

- **Cross-process hash determinism** — spawns a fresh interpreter. If the extractor and chargeback parser resolve the salt differently, every join returns zero rows and the symptom reads as missing data, not a bug.
- **PII canary** — injects a fake SSN into a real document and proves it does not come out. Stronger than asserting absence: a broken extractor emitting nothing would also pass.
- **Velocity leakage guard** — a hand-built case where a user's third transaction must aggregate the first two and not its own amount.

```
pytest                    # everything
pytest -m "not slow"      # fast unit tests only
pytest -m security        # leakage checks
```

```
pytest -k hash            # tests matching a name
```

6. Known limitations

6.1 Label coverage is the bottleneck

The COVERAGE figure in the label audit is the share of transfers with a *confirmed* verdict. Everything else is assumed legitimate. Low coverage means the model largely learns the existing rules. Partner chargeback data is the highest-priority input to improve this — it is what makes the model a rules-improver rather than a rules-imitator.

6.2 Historical records lose Tier-2 signal

User and recipient documents are not snapshotted at transfer time, so mutable fields cannot be trusted on old records. This is deliberate — correctness over strength — and recoverable only if database oplog or backup retention reaches far enough back, which it usually does not.

6.3 Watchlist feedback is absent

Partners watchlist a user but notify only via a later declined transaction, so the transaction that *caused* the watchlisting is unlabelled. Worth asking partners to report which transaction triggered it.

6.4 description_code has two formats

Newer records carry a short numeric purpose code; older ones carry a 24-character ObjectId. The assembler buckets the legacy format and logs the ratio, warning if it exceeds half the column — bucketing the majority of a feature into one value destroys most of its signal and should be escalated rather than absorbed.

6.5 Untested paths

- XGBoost, LightGBM and Optuna API calls were written against version-shimmed interfaces but not executed in the authoring environment.
- `top_risk_factors()` needs xgboost's `pred_contribs` and has not been run.
- Optuna study persistence and resume has not been exercised. Test it deliberately: run with a few trials, interrupt, re-run, and confirm it reports resuming.

7. Operational notes

7.1 Deployment shape

Training and serving belong on **separate instances**. The training box is stopped between runs; the serving box must stay up. Compute-optimised instances suit training (no CPU credits to exhaust); burstable instances suit serving (idle most of the time, so credits accumulate).

7.2 Cost per tuning run is nearly flat

A larger instance costs more per hour but finishes proportionally sooner, so the bill barely moves across instance types. The decision is not how much to spend but how long to wait. What actually drives cost is whether the instance is stopped when idle.

7.3 Reading from MongoDB directly

Prefer streaming from the database over exporting raw JSON to disk. `extract_collection()` takes any iterable of dicts, so a pymongo cursor substitutes for a file with almost no change — and raw PII then never lands in a new location.

```
docs = db["transfers_safe"].find({}, batch_size=1000)   # a restricted view
df = extract_collection(docs, "transfer")
```

Pair this with a read-only database user restricted to a **view** that excludes the never-needed sensitive fields. The code allowlist is one layer; a server-side view that does not expose the fields at all is a stronger one, because it does not depend on the pipeline being correct.

7.4 Retraining

Fraud patterns drift faster than most ML applications because the adversary adapts. Monthly retraining is a reasonable baseline; drift-triggered retraining is better. The signal to watch is the flag rate: if a model that historically flagged 2% of transactions starts flagging 6%, something has shifted.